CNN卷積神經網路

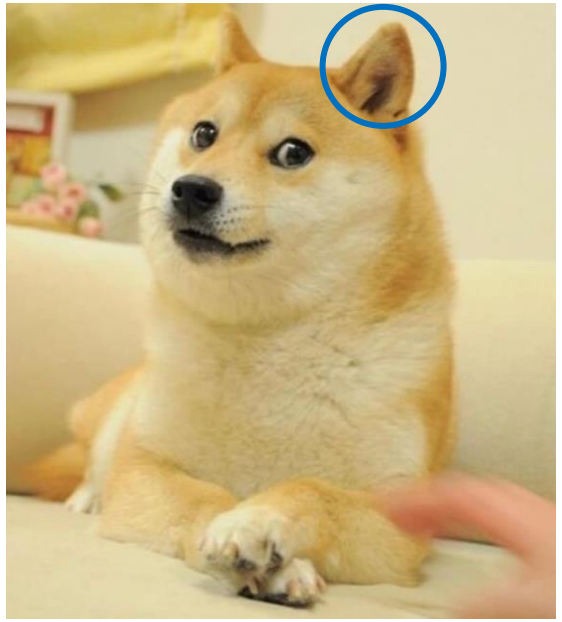
目錄

- 影像如何被計算
- CNN簡介
- 建立簡易CNN Model
  - Ex1_LeNet
- 練習
  - AlexNet
  - VGG16

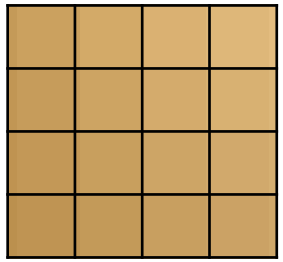
CNN怎麼來的？

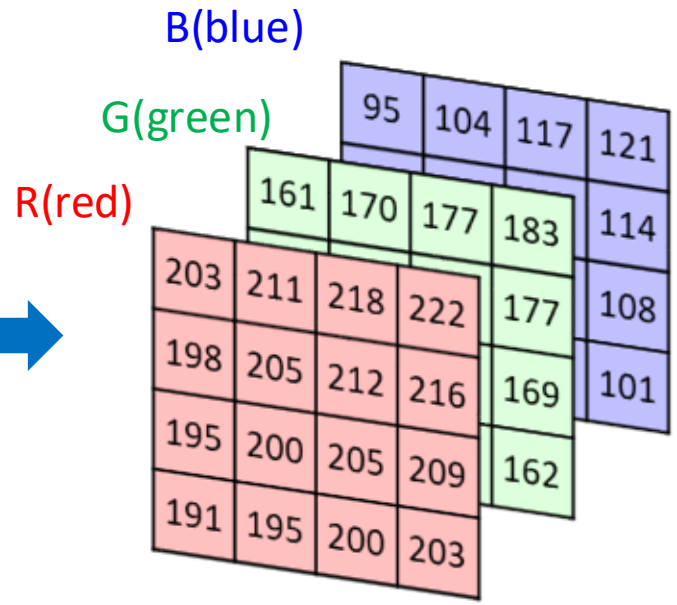
- CNN 最早的靈感來源於兩位神經生理學家 **Hubel & Wiesel**。他們在研究貓的視覺皮層時發現：
- **局部感知：** 視覺神經細胞並非對整張圖片都有反應，而是只對特定區域（感受野，Receptive Field）有反應。
- **特徵分工：** 有些細胞對「特定方向的線條」敏感，有些則對「動作」敏感。
- 這項研究證明了視覺是**由簡入繁**的階層式處理，這成為了 CNN「卷積核」設計的生物學基礎。

影像如何被計算



(4*4*3)



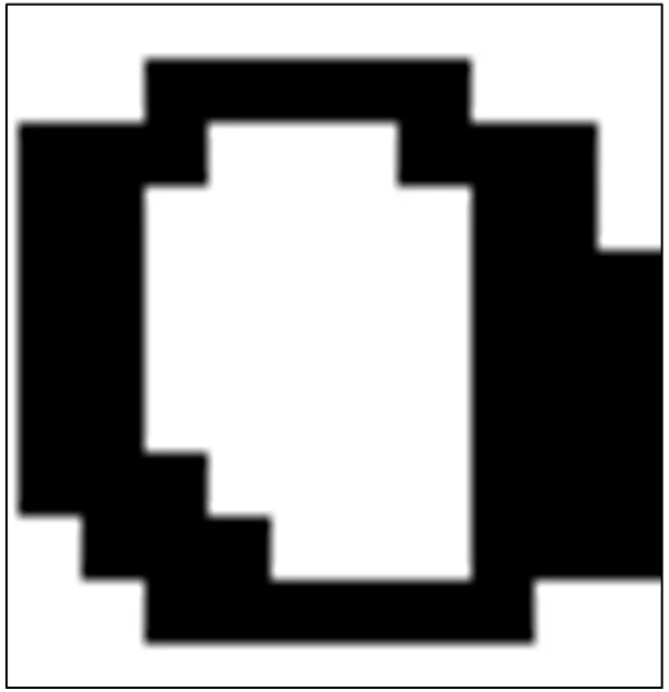

B(blue)
G(green)
R(red)
95 104 117 121
114
108
101
161 170 177 183
177
169
162
203 211 218 222
198 205 212 216
195 200 205 209
191 195 200 203

CNN簡介

現今的數位影像主要是由pixel組成，解析度= w * h

Ex. 八百萬3,360 x 2,460=8,265,600

原始影像

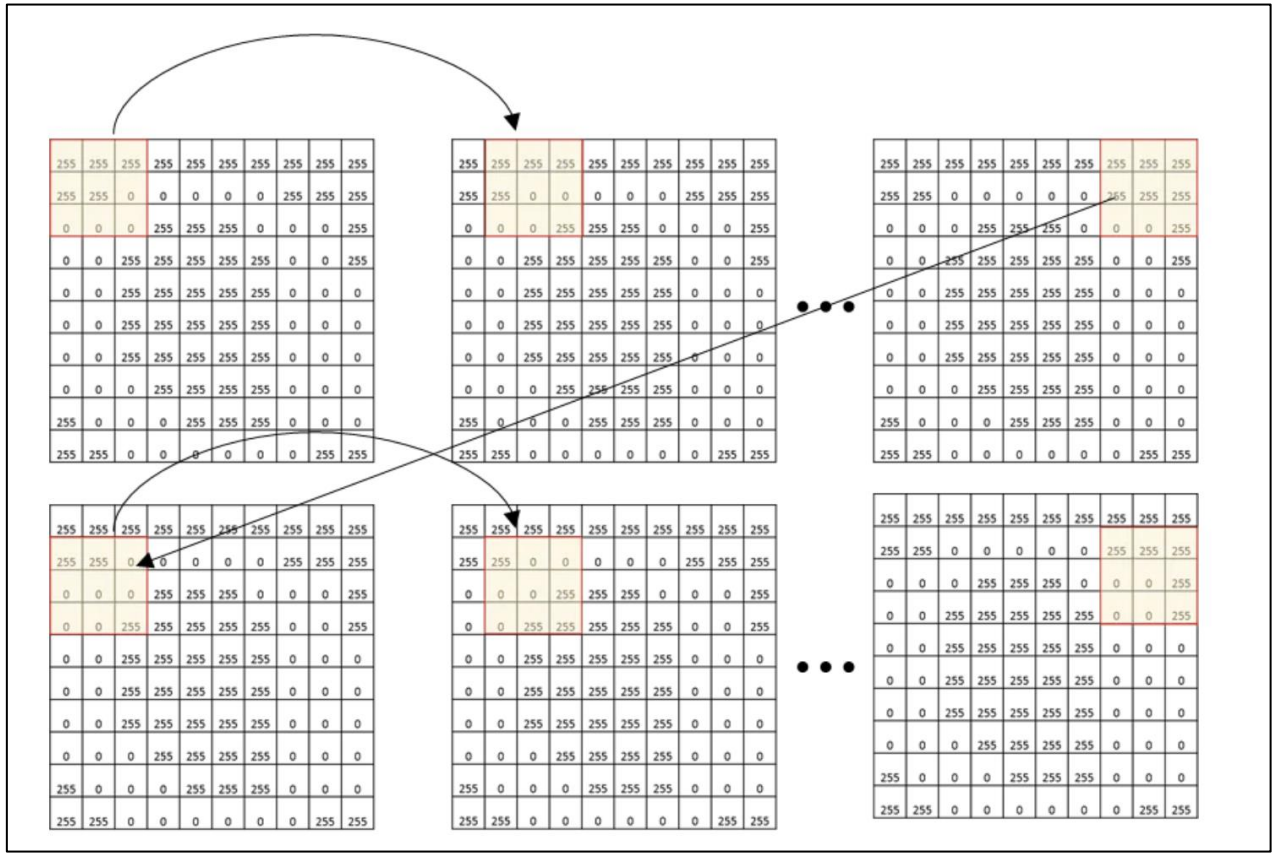


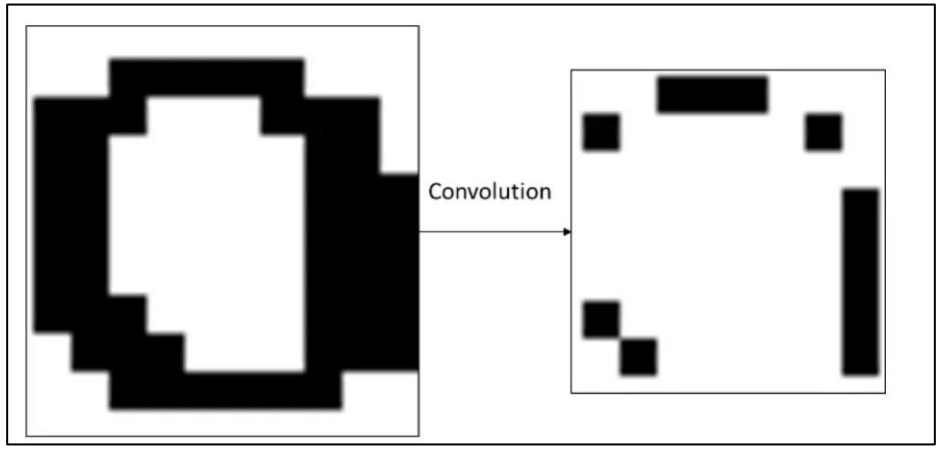
數位化的值

255	255	255	255	255	255	255	255	255	255
255	255	0	0	0	0	0	255	255	255
0	0	0	255	255	255	0	0	0	255
0	0	255	255	255	255	255	0	0	255
0	0	255	255	255	255	255	0	0	0
0	0	255	255	255	255	255	0	0	0
0	0	255	255	255	255	255	0	0	0
0	0	0	255	255	255	255	0	0	0
255	0	0	0	255	255	255	0	0	0
255	255	0	0	0	0	0	0	255	255

CNN簡介

回顧影像處理的卷積運算：



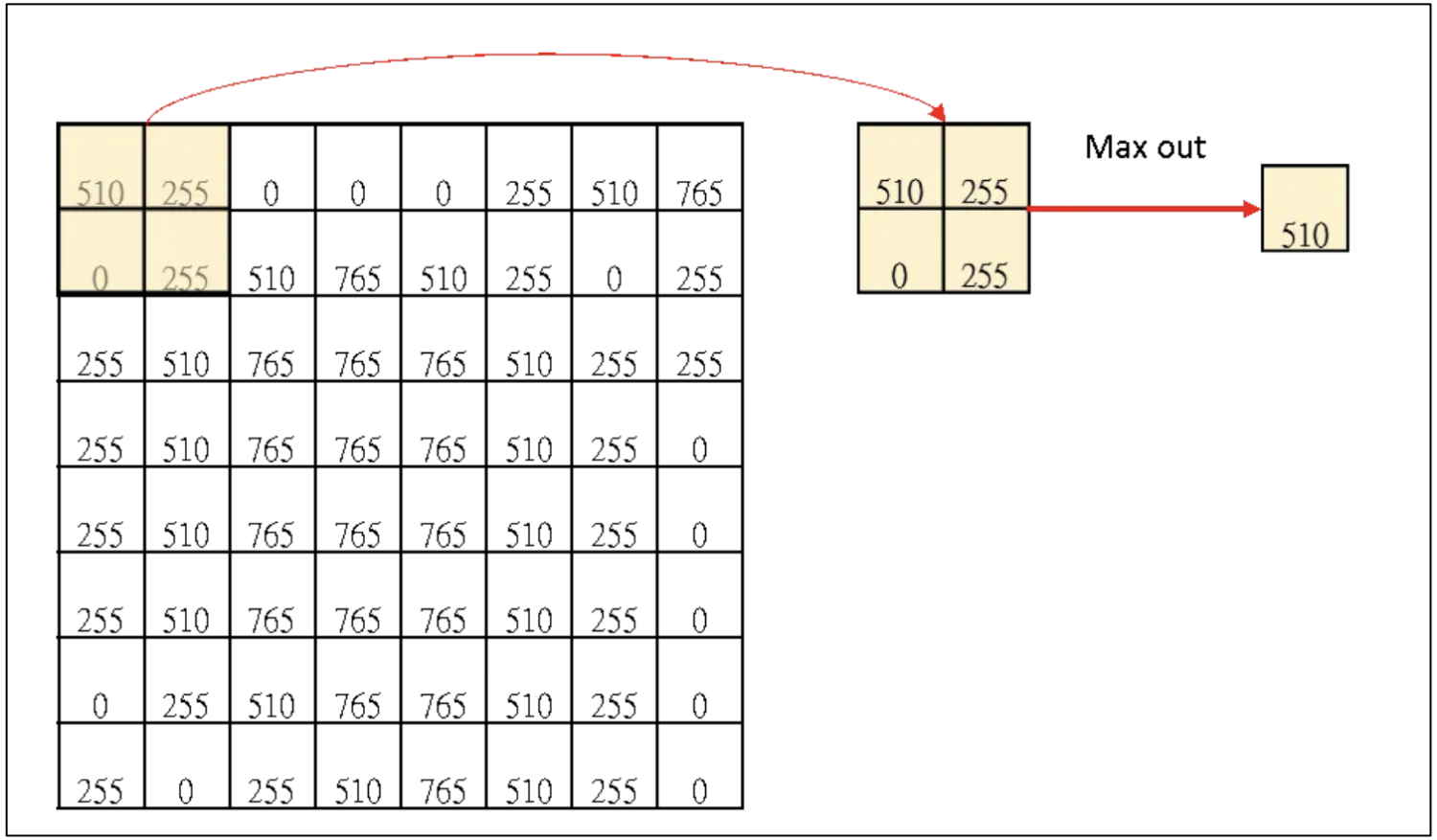


https://chih-sheng-huang821.medium.com/%E5%8D%B7%E7%A9%8D%E7%A5%9E%E7%B6%93%E7%B6%B2%E8%B7%AF-convolutional-neural-network-cnn-%E5%8D%B7%E7%A9%8D%E9%81%8B%E7%AE%97-%E6%B1%A0%E5%8C%96%E9%81%8B%E7%AE%97-856330c2b703

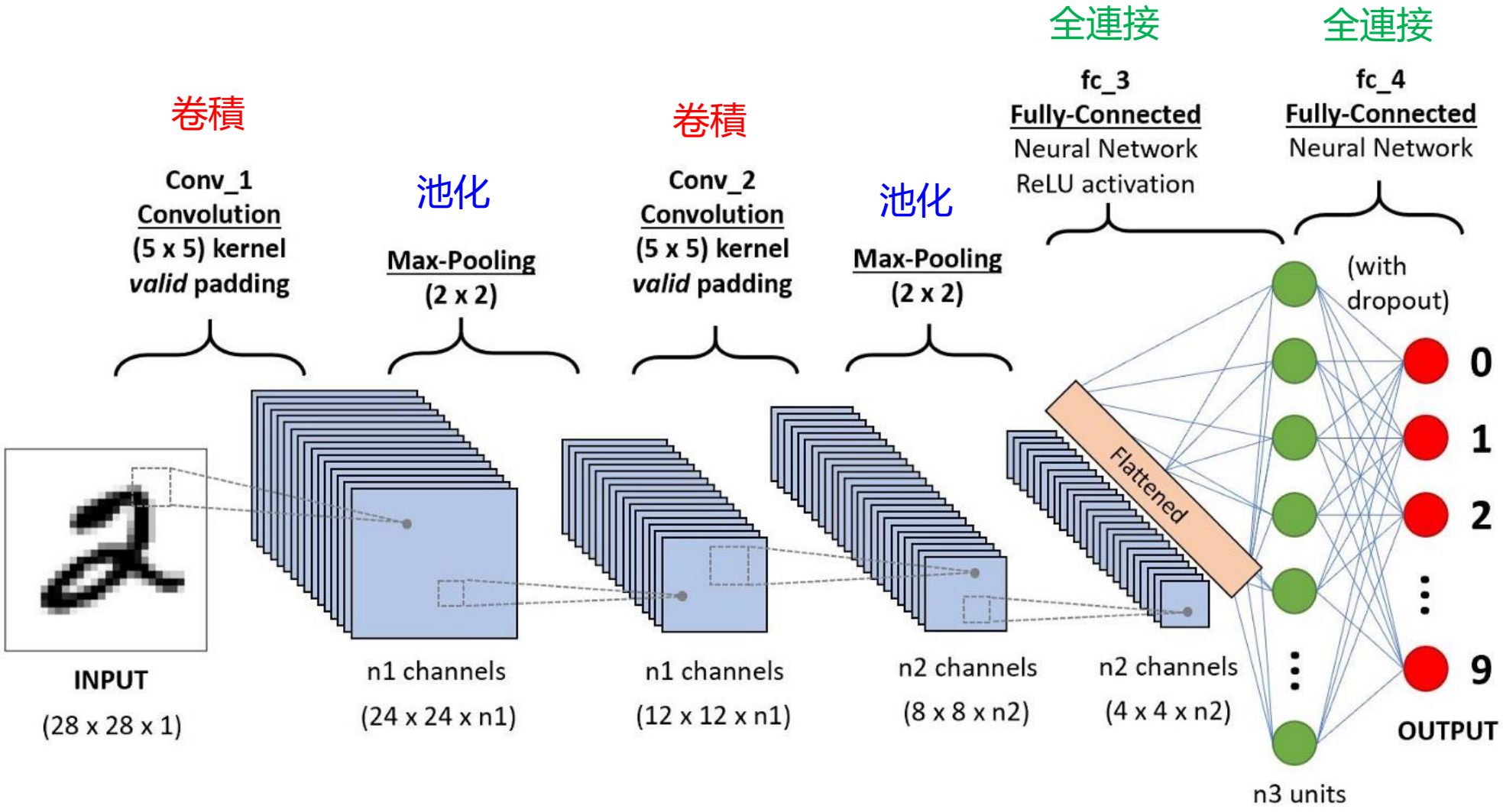
CNN簡介

池化－讓特徵圖尺寸大幅降低：

目的：減少資料量、保留重要資訊



CNN簡介


全連接
全連接
卷積
卷積
池化
池化
Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with
dropout)
Flattened
0
1
2
9
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
n3 units
OUTPUT

CNN簡介

1998 年發表的 **LeNet-5** 是第一個產生商業價值的 CNN 模型，被廣泛用於美國郵政系統的支票手寫數字識別（MNIST 數據集）。LeNet-5 確立了現代 CNN 的標準組件：

- **卷積層（Convolution）**：利用權重共享（Weight Sharing）大幅減少參數。
- **池化層（Pooling）**：降低維度並增加特徵的魯棒性。
- **全連接層（Fully Connected）**：進行最後的分類。

卷積(Convolution)

透過kernel找尋特定特徵

6	8	73	73	73	62
4	5	32	65	74	41
54	12	43	235	65	57
14	32	32	71	34	37
74	43	54	63	86	71
31	62	72	74	51	37

輸入

6x2+8x1+4x4+5x3=52

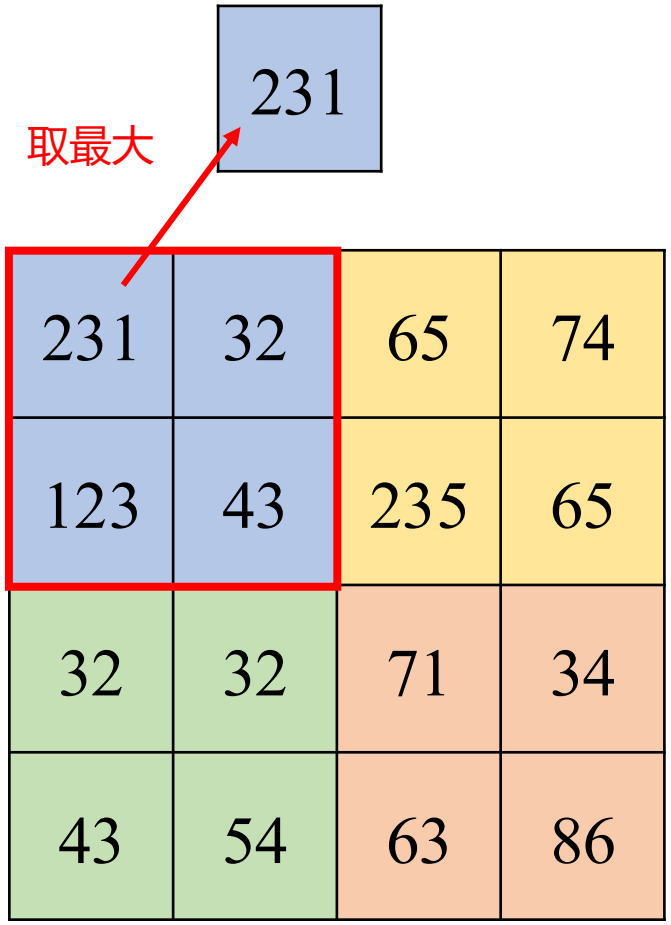
2	1
4	3

Kernel 或稱 Filter

52				

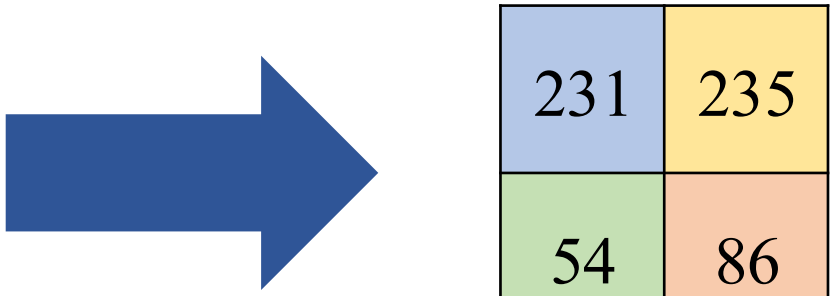
輸出

池化(Pooling)



池化前

挑選特徵最強的地方，減少之後的運算量



池化後

填充(Padding)

避免做卷積時步伐太大，直接撞牆，無法算到邊緣

231	32	65	74
123	43	235	65
32	32	71	34
43	54	63	86

Padding前

0	0	0	0	0	0
0	231	32	65	74	0
0	123	43	235	65	0
0	32	32	71	34	0
0	43	54	63	86	0
0	0	0	0	0	0

Padding後

EX1_LeNet(import)

```
import torch
import torch.nn.functional as F
import torchvision.datasets as datasets
import torchvision.transforms as transforms
from torch import optim
from torch import nn
from torch.utils.data import DataLoader
from tqdm import tqdm
```

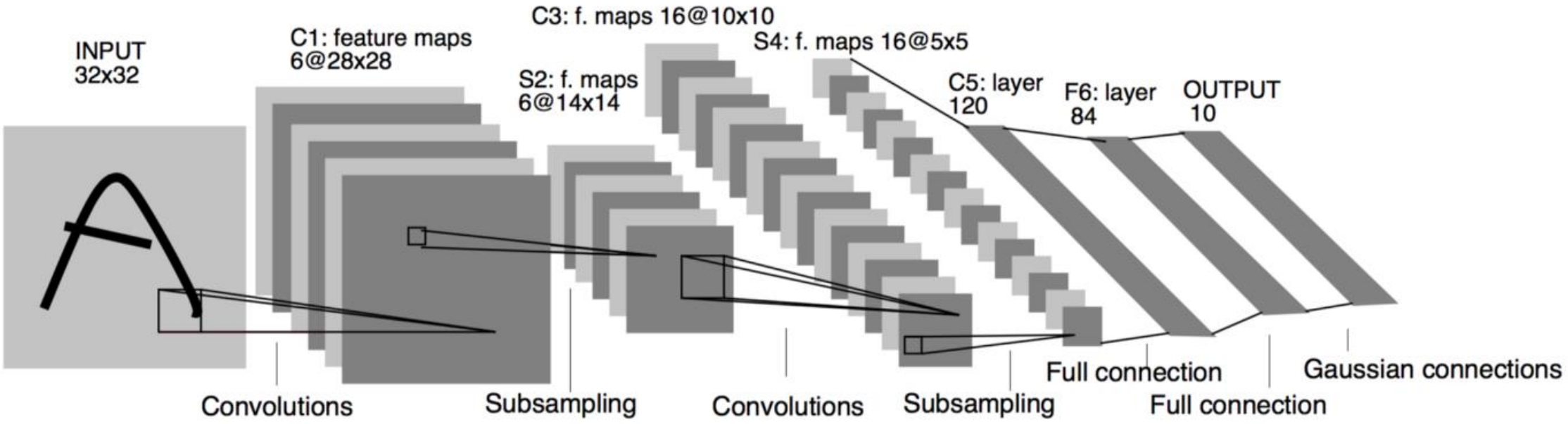
EX1_ LeNet(載入資料)

```
transform = transforms.Compose([
            transforms.Resize((32, 32)),
            transforms.ToTensor()
            ])
# Load Data
train_dataset = datasets.MNIST(root="dataset/", train=True, transform=transform, download=True)
test_dataset = datasets.MNIST(root="dataset/", train=False, transform=transform, download=True)
train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(dataset=test_dataset, batch_size=64, shuffle=True)
```

由於手寫字載下來只會是28x28，
但論文用的照片是32x32所以我們要resize。

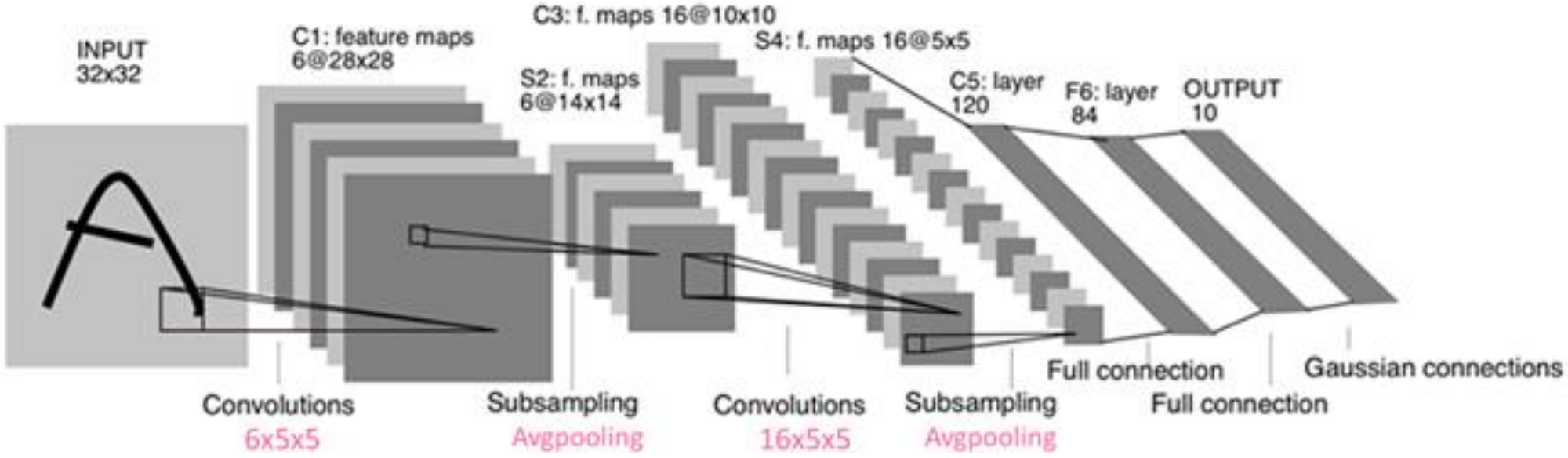
跟上次一樣64張圖打包成一個batch

EX1_LeNet(論文原始圖)



LeNet論文: http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

EX1_ LeNet(定義模型)



$$\text{output width} = \frac{W - F_w + 2P}{S_w} + 1$$

$$28 = \frac{32 - F_w + 0}{1} + 1$$

$$F_w = \boxed{5}$$

不是RGB，只有黑白所以是1

輸出想要幾片就寫多少

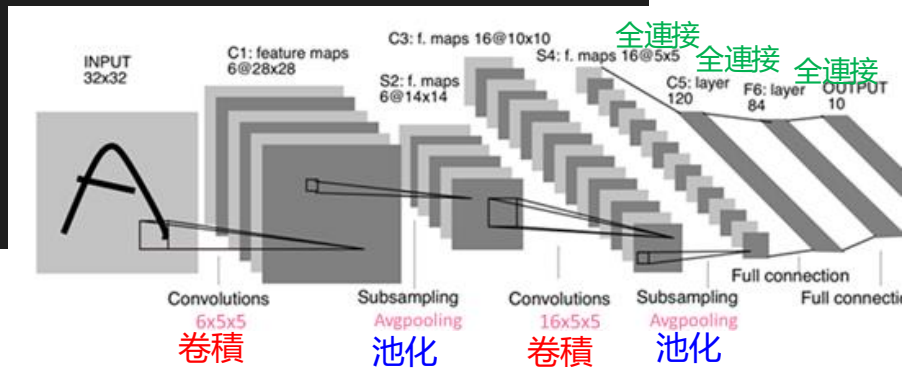
5x5的大小去滑窗

做完一次往右1格

```
self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1)
```

EX1_ LeNet(基礎架構)

```
class CNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super(CNN, self).__init__()


        定義基本網路層


    def forward(self, x):


        定義網路傳遞鏈



        return x
```



EX1_ LeNet(定義模型)

```
class CNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)
        self.fc1 = nn.Linear(in_features=16*5*5, out_features=120)
        self.fc2 = nn.Linear(in_features=120, out_features=84)
        self.fc3 = nn.Linear(in_features=84, out_features=10)

    def forward(self, x):
        x = torch.sigmoid(self.conv1(x))  #輸出=[64, 6, 28, 28]
        x = F.avg_pool2d(x, kernel_size=2, stride=2)  #輸出=[64, 6, 14, 14]
        x = torch.sigmoid(self.conv2(x))  #輸出=[64, 16, 10, 10]
        x = F.avg_pool2d(x, kernel_size=2, stride=2)  #輸出=[64, 16, 5, 5]
        x = torch.flatten(x, 1)  #輸出=[64, 400]
        x = torch.sigmoid(self.fc1(x))  #輸出=[64, 120]
        x = torch.sigmoid(self.fc2(x))  #輸出=[64, 84]
        x = self.fc3(x)  #輸出=[64, 10]
        return x
```

兩種卷積

三種全連接

卷積

池化

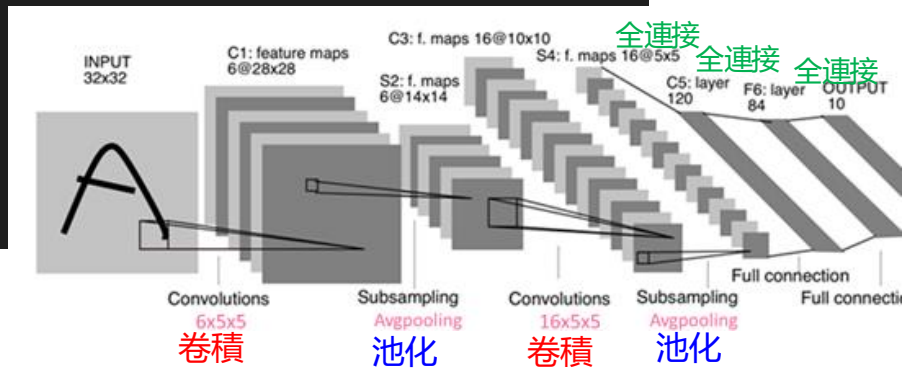
卷積

池化

全連接

全連接

全連接



EX1_ LeNet(初始化模型、loss、optim)

```
# Initialize network
model = CNN(in_channels=1, num_classes=10).to("cuda")
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=3e-4)
```

Tesla首席技術長告訴你如何調參數

EX1_ LeNet(開始訓練)

```
# Train Network
model.train()
for epoch in range(3):
    for batch_idx, (data, targets) in enumerate(tqdm(train_loader)):
        # Get data to cuda if possible
        data = data.to("cuda")
        targets = targets.to("cuda")


        # forward
        scores = model(data)

        loss = criterion(scores, targets)

        # backward
        optimizer.zero_grad()
        loss.backward()

        # gradient descent or adam step
        optimizer.step()
```

مدّقت 進度條 (發音:tqddm 嘎夸達麻)

跑一次

算誤差

清除上一次的grad

計算這一次的grad

更新權重

EX1_ LeNet(驗證)

```
num_correct = 0
num_samples = 0
model.eval()

with torch.no_grad():
    for x, y in test_loader:
        x = x.to("cuda")
        y = y.to("cuda")

        scores = model(x)
        _, predictions = scores.max(1)
        num_correct += (predictions == y).sum()
        num_samples += predictions.size(0)

print(num_correct/num_samples)
```

```
tensor(0.9049, device='cuda:0')
```

練習AlexNet

當初是因為運算量不足所以才拆開跑，可以只用一顆GPU跑

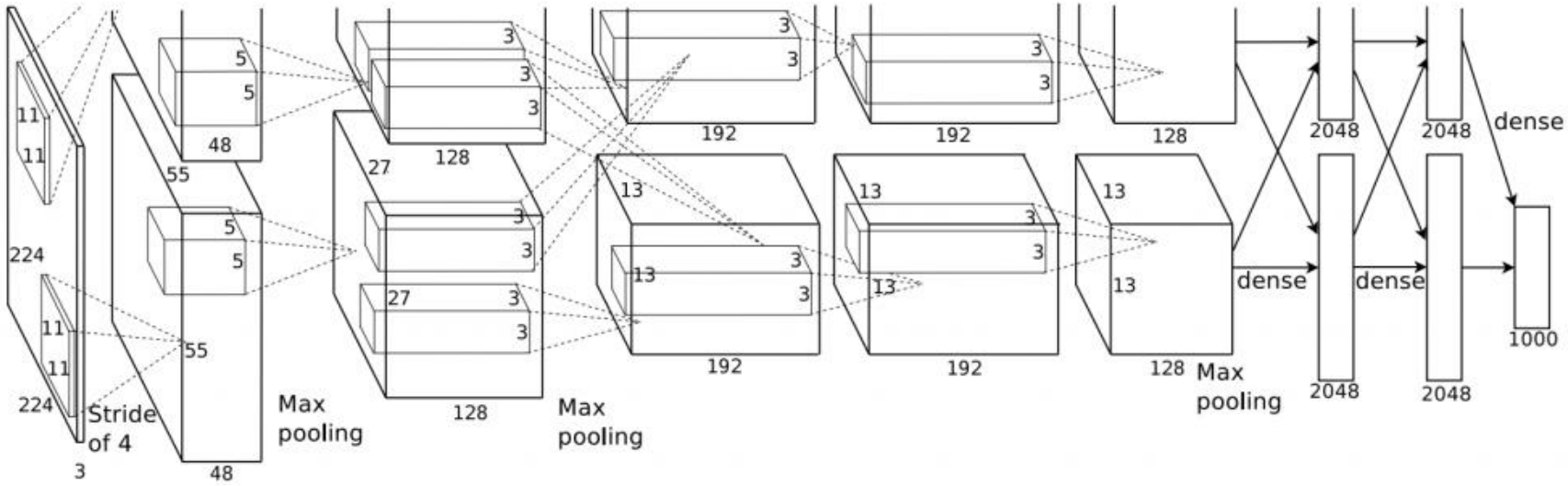


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet論文:https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

參考

Pytorch建立簡易CNN : https://www.youtube.com/watch?v=wnK3uWv_WkU

LeNet論文: http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

AlexNet論文: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

VGG論文: https://arxiv.org/pdf/1409.1556.pdf